

Phần 1: ORM Cơ bản và JPA Độc lập

1.1 Giới thiệu về ORM (Object-Relational Mapping)

ORM là một kỹ thuật lập trình cho phép ánh xạ dữ liệu giữa hệ thống đối tượng (object-oriented programming language) và hệ thống cơ sở dữ liệu quan hệ (relational database). Mục tiêu chính của ORM là loại bỏ sự cần thiết phải viết các câu lệnh SQL trực tiếp trong ứng dụng, giúp nhà phát triển làm việc với cơ sở dữ liệu thông qua các đối tượng Java.

Lợi ích của ORM:

- **Tăng năng suất:** Giảm thời gian viết code SQL lặp đi lặp lại.
- **Dễ bảo trì:** Code dễ đọc và dễ quản lý hơn.
- **Độc lập với cơ sở dữ liệu:** Có thể dễ dàng chuyển đổi giữa các hệ quản trị cơ sở dữ liệu khác nhau (ví dụ: MySQL, PostgreSQL, Oracle) mà không cần thay đổi nhiều code.
- **An toàn hơn:** Giảm thiểu các lỗi liên quan đến SQL Injection.

1.2 JPA (Java Persistence API)

JPA là một đặc tả (specification) của Java, định nghĩa cách các ứng dụng Java có thể tương tác với cơ sở dữ liệu quan hệ. JPA không phải là một triển khai cụ thể, mà là một tập hợp các giao diện và annotation. Các nhà cung cấp (vendors) như Hibernate, EclipseLink, OpenJPA sẽ cung cấp các triển khai cụ thể của đặc tả JPA.

Các khái niệm cốt lõi trong JPA:

1.2.1 Entity

Entity là một POJO (Plain Old Java Object) đại diện cho một bảng trong cơ sở dữ liệu. Mỗi instance của Entity tương ứng với một hàng (row) trong bảng đó. Entity được đánh dấu bằng annotation `@Entity`.

Ví dụ:

```
Java
```

```

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;

@Entity // Đánh dấu lớp User là một Entity JPA, tương ứng với một bảng trong
cơ sở dữ liệu.
public class User {

    @Id // Đánh dấu trường 'id' là khóa chính của Entity.
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Cấu hình cách sinh
    giá trị cho khóa chính. IDENTITY sử dụng cơ chế auto-increment của cơ sở dữ
    liệu.
    private Long id;
    private String name;
    private String email;

    // Constructors, getters, setters (các phương thức khởi tạo, lấy/đặt giá
    trị cho các trường)
    // Các phương thức này giúp truy cập và thao tác với dữ liệu của đối
    tượng User.
}

```

Giải thích: Đoạn code trên định nghĩa một Entity `User` đơn giản. Lớp này sẽ được ánh xạ tới một bảng `User` trong cơ sở dữ liệu. Trường `id` được đánh dấu là khóa chính và giá trị của nó sẽ được tự động sinh ra bởi cơ sở dữ liệu.

1.2.2 Persistence Context

Persistence Context là một tập hợp các Entity instances mà tại một thời điểm nhất định, chúng được quản lý bởi một `EntityManager`. Nó hoạt động như một bộ đệm (cache) cho các Entity, giúp theo dõi các thay đổi và đồng bộ hóa chúng với cơ sở dữ liệu.

Các trạng thái của Entity trong Persistence Context:

- **New (Transient):** Đối tượng mới được tạo, chưa được liên kết với `EntityManager` và chưa có trong cơ sở dữ liệu.
- **Managed (Persistent):** Đối tượng đã được liên kết với `EntityManager` và được quản lý bởi Persistence Context. Mọi thay đổi trên đối tượng này sẽ được theo dõi và đồng bộ hóa với cơ sở dữ liệu.

- **Detached:** Đối tượng đã từng được quản lý bởi `EntityManager` nhưng hiện tại không còn được quản lý nữa (ví dụ: sau khi `EntityManager` đóng hoặc `clear()`).
- **Removed:** Đối tượng đã được đánh dấu để xóa khỏi cơ sở dữ liệu.

1.2.3 EntityManager

`EntityManager` là giao diện chính trong JPA để tương tác với Persistence Context. Nó cung cấp các phương thức để thực hiện các thao tác CRUD (Create, Read, Update, Delete) trên các Entity.

Các phương thức quan trọng của `EntityManager` :

- `persist(Object entity)` : Đưa một Entity mới vào Persistence Context (trạng thái Managed).
- `find(Class<T> entityClass, Object primaryKey)` : Tìm một Entity theo khóa chính.
- `merge(Object entity)` : Hợp nhất một Entity (có thể là Detached) vào Persistence Context. Nếu Entity đã tồn tại, nó sẽ được cập nhật; nếu không, một Entity mới sẽ được tạo.
- `remove(Object entity)` : Đánh dấu một Entity để xóa khỏi cơ sở dữ liệu.
- `flush()` : Đồng bộ hóa các thay đổi trong Persistence Context với cơ sở dữ liệu.
- `clear()` : Xóa tất cả các Entity khỏi Persistence Context, đưa chúng về trạng thái Detached.

1.2.4 JPA Query

JPA cung cấp các cách để truy vấn dữ liệu từ cơ sở dữ liệu:

- **JPQL (Java Persistence Query Language):** Ngôn ngữ truy vấn hướng đối tượng, tương tự SQL nhưng làm việc với các Entity và thuộc tính của chúng thay vì bảng và cột. JPQL độc lập với cơ sở dữ liệu.
- **Criteria API:** Một API lập trình cho phép xây dựng các truy vấn động và an toàn kiểu (type-safe) trong Java. Rất hữu ích cho các truy vấn phức tạp hoặc khi các điều kiện truy vấn được xây dựng dựa trên đầu vào của người dùng.

- **Native SQL:** Cho phép thực thi các câu lệnh SQL thuần túy. Hữu ích khi cần sử dụng các tính năng cụ thể của cơ sở dữ liệu hoặc khi JPQL/Criteria API không đáp ứng được.

1.3 Vòng đời của Entity (Entity Lifecycle)

Entity trong JPA trải qua các trạng thái khác nhau trong suốt vòng đời của nó:

1. **New (Transient):** Entity mới được tạo bằng từ khóa `new`, chưa được liên kết với `EntityManager` và chưa có trong cơ sở dữ liệu.
2. **Managed (Persistent):** Entity được liên kết với một Persistence Context và được quản lý bởi `EntityManager`. Mọi thay đổi trên Entity này sẽ được theo dõi và đồng bộ hóa với cơ sở dữ liệu khi transaction commit hoặc khi `flush()` được gọi.
 - Chuyển từ New sang Managed:
 - Chuyển từ Detached sang Managed:
 - Chuyển từ cơ sở dữ liệu sang Managed:
3. **Detached:** Entity đã từng được quản lý bởi `EntityManager` nhưng hiện tại không còn được quản lý nữa. Điều này xảy ra khi `EntityManager` đóng, `clear()` được gọi, hoặc Entity được serialize và deserialize.
4. **Removed:** Entity được đánh dấu để xóa khỏi cơ sở dữ liệu. Entity ở trạng thái Removed vẫn được quản lý bởi Persistence Context cho đến khi transaction commit, lúc đó nó sẽ bị xóa khỏi cơ sở dữ liệu và trở thành New (Transient).

Hiểu rõ vòng đời của Entity là rất quan trọng để quản lý dữ liệu hiệu quả và tránh các lỗi không mong muốn.